
PETalytics: Progress Report

Owen Poyner

1010863335, owen.poyner@mail.utoronto.ca

github.com/OwenPoyner/PETalytics

Project Description

PETalytics uses PETadex (1), a new, massive database of over a billion (likely-)plastic-degrading enzymes to create a modular framework (see Fig. 1) for predicting polyethylene terephthalate-degrading enzyme (PETase) properties. It starts with raw amino acid sequences, the physical peptide chains that make up proteins, that are fed into a pretrained ESM-2 (2), a state-of-the-art protein language model (pLM) that is trained on billions of general proteins (not just catalytically active enzymes like PETases). ESM-2 embeddings are then collected and given to a variational autoencoder (VAE) that is only trained on PETadex sequences and therefore becomes specialized at encoding (and processing) information on likely-PETase proteins. The latent vectors from the VAE are finally used as inputs into relatively small prediction "head" stages that can be trained on relatively small, labelled datasets on any characterizable feature of a PETase, such as activity, temperature preference, specificity, and more.

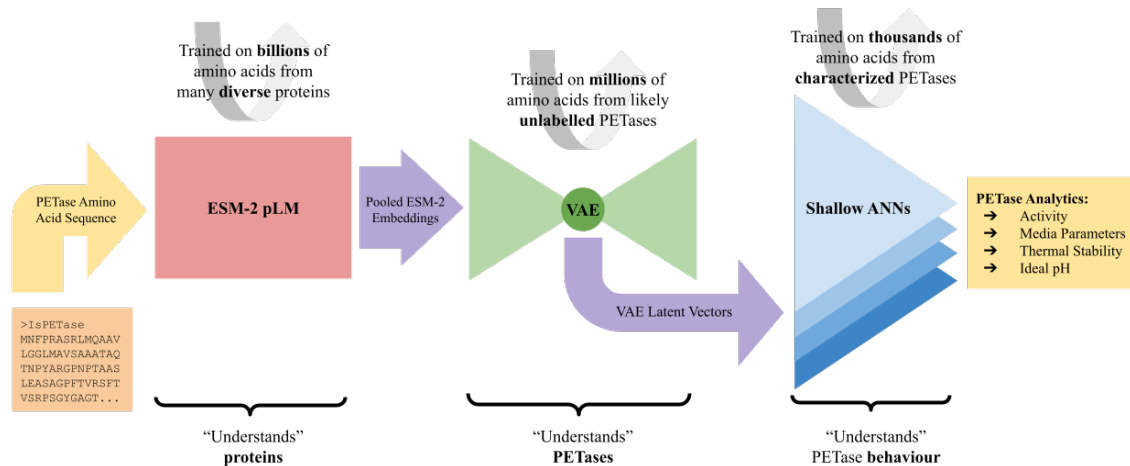


Figure 1: Overall model architecture summarizing the role of each stage of PETalytics and the data they are trained on.

Data Processing

Some of my data processing is currently limited by the fact that the PETadex is currently being recreated with new reference sequences that were missing in the original construction. While this does mean I cannot make any fixed references to the database, the new database will be much larger and cover more industry-relevant PETases that will help PETalytic's performance.

The VAE feasibility dataset was collected from a sample of 50 000 pregenerated 90% cluster centroids (out of 19.8 million available at the time). Using 90% centroids means that the full sequence space of the database was clustered into groups of similar sequences, and a single representative sequence for each cluster was used. This increases diversity in the dataset and prevents overfitting to proteins that just happened to be sequenced more in the Logan (3) inputs that were queried to create PETadex.

These were further cleaned by removing any sequences over 1024 amino acids (1024AA) in length as they are computationally costly outliers (most known PETases are about 300AA).

esm2_t30_150M_UR50D (4) was used to generate ESM-2 embeddings to balance embedding quality with performance (generating about 50 sequences per second on an A100 80GB) and mean-pooled (5) to create a 640-dimensional embedding for each sequence. The modular prediction heads of PETalytics require labelled input data for each sequence. Finding suitable labelled sets for this is extremely difficult, as it is generally bad practice to collate wet lab results between labs, let alone different assays. However, PETadex includes a halo assay (3) .csv file. I was able to cross-reference the multiple accession systems used to find the base amino acids of all the proteins tested and used them to remove duplicates and isolate labelled halo assay data for 254 different proteins at 120 hours with 25mM BHET for their assay conditions.

While this extremely lean dataset is a current challenge in the project, I am now working with the PETadex creators to design and publish a paper on a schema to allow mass literature scraping by both humans and machine learning models to exponentially expand the labelled data contained in the database. Ideally, I will be able to use some of this scraped data to create better prediction heads for the final report.

VAE Preprocessor

For the VAE preprocessor, I used a symmetric encoder/decoder with two hidden layers and a variable number of neurons in each hidden layer and the latent space (all three variables are tuned hyperparameters). The loss function was a mean-squared error function for the reconstruction and a KL-divergence term with beta linearly ramping from 0 to 1 (beta=1 being standard for VAEs) over 2 to 8 epochs (a tuned hyperparameter) during training to stop the model fitting to the KL-divergence loss before any reconstruction can be learned (6). A 70/15/15 conventional training/validation/testing split was used over 50 000 ESM-2 embeddings. Optuna (7), a Bayesian hyperparameter optimization tool, was used over 20 15-epoch trials to optimize the previous hyperparameters as well as learning rate and batch size. Hyperparameter optimization resulted in a variational encoder with structure and neuron counts following 640 (input dimension)->256->128->36 (latent dimension), with the inverse being the decoder. It should be noted that the latent dimension was permitted to go up to 64 dimensions, but 36 was found to be optimal (see Fig. 2).

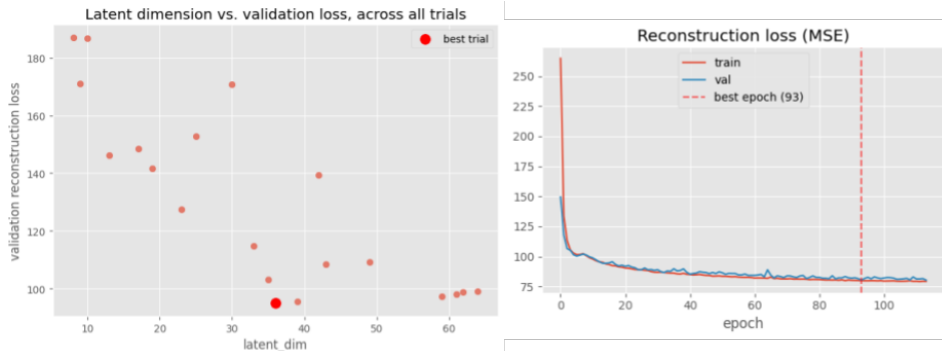


Figure 2: (Left) Latent space dimension vs loss during hyperparameter tuning (other hyperparameters also varied in parallel). (Right) Training curve with mean-squared reconstruction loss.

The final training run resulted in a model with a test set MSE reconstruction loss of 80.5, only slightly more than the validation set's 80.3, and an average coefficient of determination (R^2) of 0.87. The baseline model for this step was chosen to be a principal component analysis (PCA) over the same training set in 36 dimensions, to match the VAE latent space (8). Inverse PCA reconstructions on the test set received an R^2 of 0.81, lower than the VAE's, showing that even with this reduced dataset (50 000 versus 19.8 million), machine learning outperforms the baseline model. A 2D PCA of the latent VAE vectors, shaded according to sequence length, interestingly showed sequence length changing along one of the primary directions, implying that length information is being carried through both ESM-2 and the VAE despite never being explicitly given.

Modular Prediction Head

The prediction head of the model is designed to be small, easily trainable over small datasets and modular. To test this, I constructed a standard ANN with two same-sized hidden layers of variable size (8, 16, or 32 neurons), an input layer that scales to the input dimension of the input data, and a single output neuron. This is done because I use two baseline models for this stage that are meant to be similar in raw complexity to the prediction head, but one acts on raw, extremely high-dimensional (10 000+ depending on the largest sequence length) one-hot unprocessed amino acid sequence inputs and another on the 640-dimensional mean-pooled ESM-2 embeddings. These models are all trained using 5-fold cross-validation to maximize coverage of the 254 input data points (15% reserved for the test set), neuron dropout and weight decay to reduce overfitting. All hyperparameters were optimized using Optuna again over 25 60-epoch trials, with final training runs set to 300 epochs with a patience of 20 epochs to save training time. These models acted as regressors to maximize dataset usage (a yes/no activity classifier would have only 19 "yes" labels out of 254 total), and their performance on the test set was measured as their coefficient of determination (R^2) from predicted halo intensity (the output of the halo assay) to the actual intensity. Figure 3 shows training curves and test set prediction plots for each model.

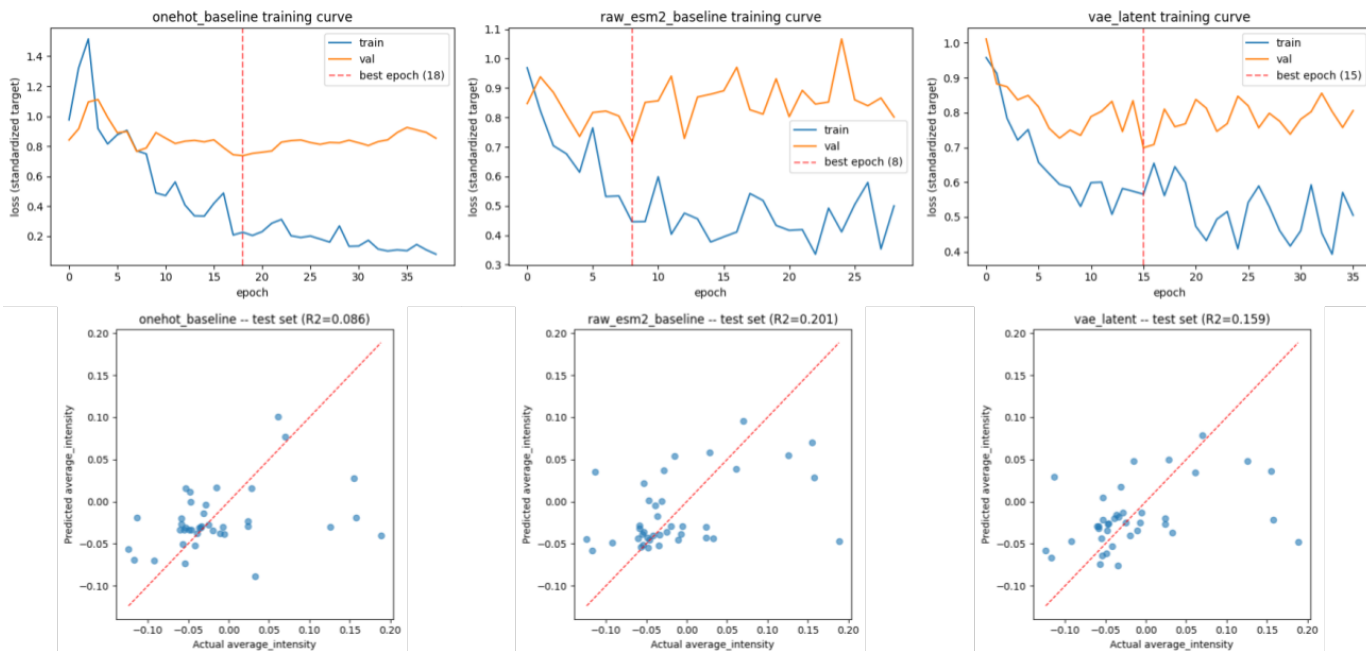


Figure 3: (Top) Training curves for the one-hot baseline, ESM-2 baseline, and actual VAE prediction head showing selected (best) model checkpoint and MSE training/validation loss. (Bottom) Prediction versus actual halo assay intensities for the test set for each model and coefficients of determination.

While the one-hot baseline and VAE model each selected 32 neurons for their hidden layers in tuning, the ESM-2 baseline settled at 8 neurons, resulting in a total of 325 600 parameters for the one-hot baseline, 5 192 for the ESM-2 baseline, and 2 208 for the VAE model. The one-hot baseline performed the worst with $R^2 = 0.09$; the ESM-2 baseline at $R^2 = 0.20$ outperformed the VAE model's $R^2 = 0.16$. While this outcome could be a symptom of reduced parameters, it could also be a result of an under-trained VAE with too small a latent space. I will be testing this more as the project goes on and I have access to larger labelled sets of data, as overfitting was by far the biggest quality/challenge I saw in these prediction models.

References

- [1] T. P. contributors, “Petadex: an open atlas of plastic-degrading enzymes,” 2026, v0.4 (Apr 2026). [Online]. Available: <https://petadex.net>
- [2] Z. Lin, H. Akin, R. Rao, B. Hie, Z. Zhu, W. Lu, N. Smetanin, R. Verkuil, O. Kabeli, Y. Shmueli, A. D. S. Costa, M. Fazel-Zarandi, T. Sercu, S. Candido, and A. Rives, “Evolutionary-scale prediction of atomic-level protein structure with a language model,” *Science*, vol. 379, no. 6637, pp. 1123–1130, 3 2023. [Online]. Available: <https://doi.org/10.1126/science.ade2574>
- [3] R. Chikhi, T. Lemane, R. Loll-Kripplleber, M. Montoliu-Nerin, B. Raffestin, A. P. Camargo, C. J. Miller, M. B. Fiamenghi, D. P. Agostinho, S. Majidian, G. Autric, M. Hugues, J. Lee, R. Faure, K. D. Curry, J. A. M. De Sousa, E. P. C. Rocha, D. Koslicki, P. Medvedev, P. Gupta, J. Shen, A. Morales-Tapia, K. Sihuta, P. J. Roy, G. W. Brown, R. C. Edgar, A. Korobeynikov, M. Steinegger, C. A. Lareau, P. Peterlongo, and A. Babaian, “Logan: Planetary-Scale Genome Assembly surveys life’s diversity,” *bioRxiv (Cold Spring Harbor Laboratory)*, 7 2024. [Online]. Available: <https://doi.org/10.1101/2024.07.30.605881>
- [4] Facebook. facebook/esm2_t30_150m_ur50d. [Online]. Available: https://huggingface.co/facebook/esm2_t30_150M_UR50D
- [5] D. F. Muir, P. Grosjean, M. M. Pinney, and M. J. Keiser, “Leveraging protein language model embeddings for catalytic turnover prediction of adenylate kinase orthologs in a Low-Data regime,” *ArXiv.org*, 5 2025. [Online]. Available: <http://arxiv.org/abs/2505.03066>
- [6] S. R. Bowman, L. Vilnis, O. Vinyals, A. Dai, R. Jozefowicz, and S. Bengio, “Generating Sentences from a Continuous Space,” *ACL*, pp. 10–21, 1 2016. [Online]. Available: <https://doi.org/10.18653/v1/k16-1002>
- [7] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna,” *ACM Digital Library*, pp. 2623–2631, 7 2019. [Online]. Available: <https://doi.org/10.1145/3292500.3330701>
- [8] G. E. Hinton and R. R. Salakhutdinov, “Reducing the Dimensionality of Data with Neural Networks,” *Science*, vol. 313, no. 5786, pp. 504–507, 7 2006. [Online]. Available: <https://doi.org/10.1126/science.1127647>

Note: Claude was used for most of the coding (including figure generation code) for the project so far. All writing in this report is human, and all figures and statistics have been checked for hallucinations.